

AI/ML Developer Machine Test

LangGraph Supervisor Multi-Agent System (Document Grounded)

Objective

Create AI system using:

- supervisor agents
- multi-agent orchestration
- LangGraph workflows
- document-grounded retrieval
- hallucination prevention
- structured AI outputs

The System must be based on the provided geotechnical PDF document.

Document will be provided externally.

The candidate MUST:

- use LangGraph
- implement a Supervisor Agent
- implement multiple specialized agents
- use graph state management
- answer ONLY from the provided PDF
- prevent hallucinated responses
- return structured JSON responses

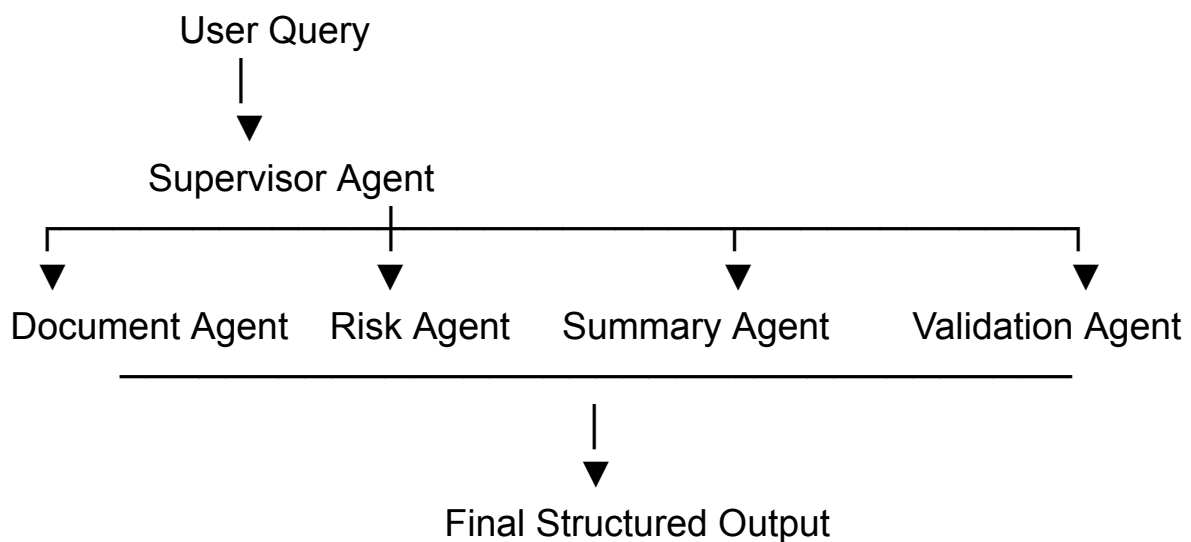
Problem Statement

Build a multi-agent AI system that can analyze and answer questions related to the uploaded geotechnical engineering report.

The system should:

- parse the PDF
- extract contextual information
- route tasks through agents
- validate responses
- return grounded answers

Required System Architecture



Mandatory Agents

1. Supervisor Agent

Responsibilities

- analyze incoming query
- decide which agents should execute
- control routing flow
- combine outputs
- manage graph state

Must Demonstrate

- conditional routing
- node orchestration
- graph transitions
- state passing

Evaluation Focus

- routing quality
- reasoning logic
- proper orchestration

2. Document Extraction Agent

Responsibilities

Extract relevant information from PDF including:

- soil layers
- groundwater details
- foundation recommendations
- risks
- measurements

- engineering observations

Example Queries

What soil types were found?

What is the groundwater level?

Expected Capability

The agent should retrieve only relevant context from the document.

3. Risk Analysis Agent

Responsibilities

Analyze extracted engineering data and identify:

- drainage risks
- groundwater risks
- settlement risks
- unstable soil risks
- construction concerns

Example Output

```
{  
  "risk_level": "medium",  
  "risks": [  
    "Secondary groundwater accumulation possible",  
    "Clay soil may create settlement issues"  
  ]  
}
```

4. Summary Agent

Responsibilities

Generate:

- concise technical summaries
- simplified explanation for non-technical users

Example Query

Summarize the site condition in simple language.

5. Validation Agent

Responsibilities

- verify response exists in document
- reject unsupported answers
- prevent hallucinations
- return confidence status

Required Behavior

If answer not found:

```
{  
  "status": "not_found",  
  "message": "Information not found in document"  
}
```

Mandatory Features

Feature 1 — PDF Processing

Candidate must:

- load PDF
- extract text
- chunk document properly
- make content searchable

Feature 2 — LangGraph Workflow

Candidate must use:

- nodes
- edges
- conditional edges
- typed state
- graph execution flow

Feature 3 — Structured Outputs

All responses must be valid JSON.

Feature 4 — Query API

Create:

POST /query

Input Example

```
{  
  "question": "What groundwater risks are mentioned in the report?"  
}
```

Expected Output Example

```
{  
  "answer": "Secondary groundwater accumulation may occur above  
impermeable clay layers.",  
  "source_section": "3.3 Vandspejlsforhold",  
  "risk_level": "medium",  
  "validated": true  
}
```

Required Technical Stack

- Python
- LangGraph
- LangChain
- GROQ API or Local LLMs
- FAISS/Chroma/Qdrant

Submission Criteria

Candidates can submit the assignment using either of the following methods:

Option 1 — Public Git Repository

Repository must include:

- complete source code
- README.md
- requirements.txt
- `.env.example`
- setup instructions
- API usage examples

Option 2 — ZIP File

ZIP must include:

- source code
- README.md
- requirements.txt
- `.env.example`